



DFA Machine Mechanism

Authors

Mennatullah Salah Mohamed

Arwa Adel Mohamed

Khaled Ibrahim Eldesoky Ahmed

Maged Hamed

CS342 Autometa and Language Theory

College of Computers and Information, Assiut University

January 2024

Contents

I	Introduction	2
I	What is the Automata theory :	2
II	What is Finite Automat:	2
III	What is DFA:	2
II	Machine design	3
I	DFA:	3
II	Machine 1	4
III	Machine 2	5
IIII	Machine Implementation	6
I	How the code is developed in details	6
A	Appendix: Project Source Codes	7
B	References	9

I Introduction

Our project is talking about Finite Automata (FA)

I What is the Automata theory :

Automata theory is a branch of computer science and mathematics that deals with abstract machines and computational problems. Automata are mathematical models used to study and understand the behavior of computational systems. One fundamental concept in automata theory is Finite Automat

II What is Finite Automat:

Finite Automata (FA) are abstract machines used to recognize patterns within strings or sequences of symbols. They operate by transitioning between various states based on inputs and can accept or reject a string based on their state transition. A Finite Automata consists of :

1. A finite set S of N states.
2. A special start state.
3. A set of final (or accepting) states.
4. A set of transitions T from one state to another

We used in this project a way to solve the problems which is called Deterministic Finite Automata (DFA) to describe our machines, but the important question is what is the (DFA)?

III What is DFA:

Now lets talk about DFA. DFAs are particularly useful for pattern matching and recognizing regular languages, making them vital in lexical analysis, compilers, text processing, and many other areas of computer science. The power of DFAs lies in their ability to model and recognize regular languages, which are languages described by regular expressions. They provide a formal framework to understand and analyze the structure of simple patterns within strings.

II Machine design

I DFA:

DFA refers to deterministic finite automata. Deterministic refers to the uniqueness of the computation. In the DFA, the machine goes to one state only for a particular input character. it is called Deterministic Automaton. As it has a finite number of states.

A DFA is represented by digraphs called state diagrams

1. The vertices represent the states.
2. The arcs labeled with an input alphabet show the transitions.
3. The initial state is denoted by an empty single incoming arc.
4. The final state is indicated by double circles.

Its graphical representation of the DFA would be like this:

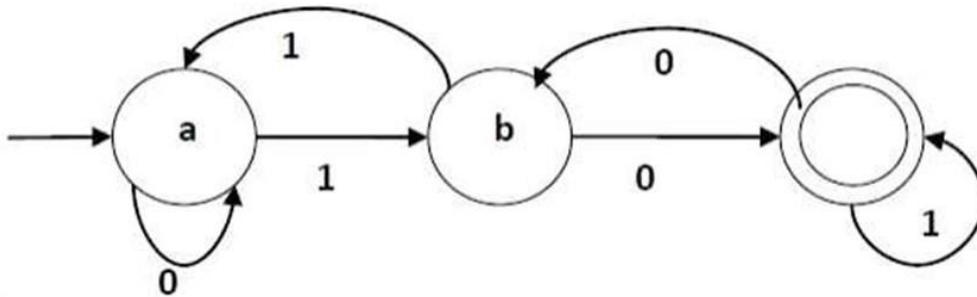


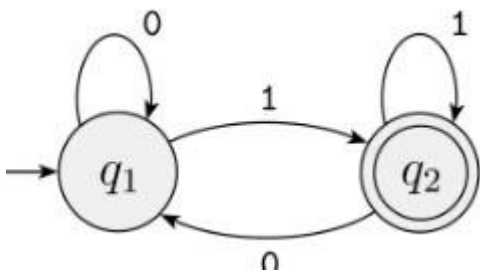
Figure 1: graphical representation of the DFA

III Machine 1:

Machine Design

We can describe the below machine (M2) formally by writing $M2 = (\{q1, q2\}, \{0, 1\}, \delta, q1, \{q2\})$

- If A is the set of all strings that machine M accepts, we say that A is the language of machine M and write • $L(M) = A$.
- $L(M2) = \{w | w \text{ ends in a } 1\}$.



- Formal definitions, Machine = $(Q, \Sigma, \delta, q0, F)$
- $Q = \{q1, q2\}$
- $\Sigma = \{0, 1\}$
- δ is described as

- q_1 is the start state
- F (final state) = $\{q_2\}$

Examples: 1 . 0011 $q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2$

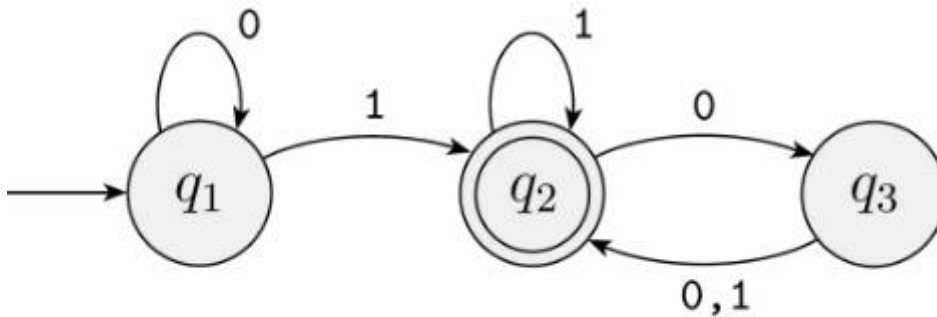
Accept 2 . 1010

$q_2 \rightarrow q_1 \rightarrow q_2 \rightarrow q_1$ **Reject**

```
Enter the number of states, number of final states and the number of trap states2
1
Enter the number of symbols in the alphabet :
2
Enter the states (initial state first)
q1
q2
Enter the final states:
q2
Enter the alphabet for the DFA:
0
1
Enter the transition for the given states:
For state q1
On input 0 to state : q1
On input 1 to state : q2
For state q2
On input 0 to state : q1
On input 1 to state : q2
The transition table you entered is :
state  0      1
q1     q1     q2
q2     q1     q2
Enter the input string( enter exit to terminate) : 0011
The input has been accepted by the DFA
```

III Machine 2:

Machine Design



Examples :

◦ 1101: $q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_3 \rightarrow q_2$

Accept ◦ 0110: $q_1 \rightarrow q_1 \rightarrow q_2 \rightarrow q_2 \rightarrow q_3$

Reject

```

2
Enter the states (initial state first)
q1
q2
q3
Enter the final states:
q2
Enter the alphabet for the DFA:
0
1
Enter the transition for the given states:
For state q1
On input 0 to state : q1
On input 1 to state : q2
For state q2
On input 0 to state : q3
On input 1 to state : q2
For state q3
On input 0 to state : q2
On input 1 to state : q2
The transition table you entered is :
state0 1
q1 q1  q2
q2 q3  q2
q3 q2  q2
Enter the input string( enter exit to terminate) : 0110
The input string has been rejected
  
```

```
2
Enter the states (initial state first)
q1
q2
q3
Enter the final states:
q2
Enter the alphabet for the DFA:
0
1
Enter the transition for the given states:
For state q1
On input 0 to state : q1
On input 1 to state : q2
For state q2
On input 0 to state : q3
On input 1 to state : q2
For state q3
On input 0 to state : q2
On input 1 to state : q2
The transition table you entered is :
state0 1
q1 q1  q2
q2 q3  q2
q3 q2  q2      Enter the input string( enter exit to terminate) : 1101
The input has been accepted by the DFA
```

III Machine Implementation

I How the code is developed in details

The code is a program that reads the number of states in the DFA from the user. It then creates an array to hold all of those states, and it also creates a map to store each state's transitions. The code begins by declaring two variables: numStates is an integer variable that will hold the number of states in the DFA, and State is a struct with three members: isAcceptState (a bool), transitions (a map), and nextState (an int). The first member of this struct will be initialized to false because it represents an initial state for which there are no transitions. The code is a simple DFA implementation. The structure to represent a state in the DFA is defined as follows: struct State bool isAcceptState; map transitions; ; The code starts by reading the alphabet used by the DFA from the user. The code then prints a prompt asking for input of each letter in the alphabet. The next line reads an integer from the user, which is how many states there are in this DFA. Then it creates a vector called "states" that will hold all of those states and initializes it with numStates. Next, it loops through every state in "states" and asks for input on what transitions should be made to move between them. The code is asking the user to enter the alphabet symbols. The code then reads the transition function for the DFA from the user. The code starts by declaring a string array called states. This is where the DFA will store all of its information about the current state and transitions for each input character. The code then declares two variables: nextState, which stores the next state to transition to after processing an input; and startState, which stores the starting state for this DFA. Next, it reads in user input from cin using a loop that goes through every possible character in order (A-Z). It then uses cout to print out what was entered so far on screen and also prints out "Enter next state for input " followed by whatever letter was just typed in. The program then enters into a while loop that keeps going until cin has no more characters left to read or until it reaches end-of-file (EOF). Inside of this while loop, it first checks if there are any remaining characters left over from before with cin » acceptStatesStr; If there are still some characters left over from before, they're stored inside of states[i].transitions[c] as well as being printed on screen again with cout « "Enter next state for input " followed by whatever letter was just typed in. After accepting all of these new The code is a DFA that has been implemented with a start state and an accept states. The code will read the user input, then look up the corresponding transition for that input in the transition list. The code will then use this information to decide what to do next. The above snippet is designed to be used as follows: Start state: Enter the start state: 1 Accept states: 2 3 The code starts by declaring a string called "acceptStatesStr" that will be used to store the states of the DFA. It then declares an array called "states" which stores all the possible states for this DFA. The code then starts looping through each state in the "states" array and sets isAcceptState to true if it's not already set, otherwise it sets isAcceptState to false. The while loop iterates until there are no more characters left in str, at which point it prints out whether or not the DFA accepts the string. The code is meant to test whether a DFA accepts a string. The code starts by initializing the start state of the DFA to be 0 and then iterating through each character in the string being tested. If the current state of the DFA is not equal to 0, it means that there are no transitions for this character, meaning that this character has been accepted by the DFA.

A Appendix: Project Source Codes

```

#include <iostream>
#include <map>
#include <vector>
#include <algorithm> using

namespace std ;

struct DFA_State
{ bool AcceptState ; map<char , int> transitions
;
};

int main()
{

    int Num_Of_States;
    cout << "Enter the number of states : " ; cin >> Num_Of_States;

    string alphabet ;
    cout << "Enter the alphabet symbols : " ; cin >> alphabet ;
    vector<DFA_State> states (Num_Of_States) ;

    for ( int eterate = 0; eterate < Num_Of_States; eterate++)
    { for ( char c : alphabet )
        {
            int nextState ;
            cout << "In state (" << eterate << "), Enter next state for input [ " << c
<< " ] : " ;
            if (!( cin >> nextState ) )
            {
                cerr << "Invalid input . Please enter integers return 1; // Exit with an
error code only .\n" ;
                states [ eterate ] . transitions [ c ] = nextState ;
            }
            an integer .\n" ;
        }
    }

    int Start_State ;
    cout << "Enter the start state : " ;
    if (!( cin >> Start_State ) )
    {
        cerr << "Invalid input for start state . Please enter return 1; // Exit with an error
code
    }

    string Accept_State ;
    cout << "Enter the accept states : " ; cin >> Accept_State ;
    for ( char c : Accept_State )
    { if ( isdigit (c) )
        {
            int acceptState = c - '0' ;
            states [ acceptState ] . AcceptState = true ;
        }
    }

```

1
2
3

4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31

32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58

```
        else
        {
            cerr << "Invalid character in accept states . Please enter digits only .\n" ; return 1; // Exit with an error code
        }
    }

    while ( true )
    { string  test ;
      cout << "Enter the string to test : " ; cin >> test ; int Current_State = Start_State ; for ( char c : test ) Current_State
        = states [ Current_State ] . transitions [ c ] ;

        cout << (( states [ Current_State ] . AcceptState )?"The DFA accepts the string .\n" : "
The DFA does not accept the string .\n") ; cout << "For Another test press
any Key, \n" ; cout << "For exit press x : " ; char ForExit ; cin >> ForExit
        ;

        if      ( ForExit == 'x ' ) break ;
    }
    return      0;
}
```

59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76

77
78
79
80
81
82
83
84
85

[link of source code](#)

B References

Introduction to the Theory of Computation book by Michael Sipser second edition